

<주제>

학생 건강 데이터를 기반으로 health_condition을 예측하는 분류 문제 대회

<파이프라인>

1. 데이터 로드 및 전처리

- isnull(). Sum(), describe()로 결측치 및 통계량 확인함

2. Train/test 분할

- Id 칼럼 제거후 stratify=y로 적용해 클래스 비율 유지한채 분할함

3. 스케일링, 및 타겟 인코딩

- map()로 문자열 라벨을 0,1,2로 매핑함

4. 모델 학습

LogisticRegression과 RandomForestClassifier를 리스트에 넣고 반복문으로 학습함, 그리고 train 정확도가 더 높은 모델을 선택함

5. 최종 제출용 저장

< 중요 코드 >

1. Train 데이터로 fit한 전처리를 test에는 transform만 적용함

2. `X_test[num_cols] = scaler.transform(X_test[num_cols])`
3. `encoded_df = ohe.transform(X_test[obj_cols])`
4. `X_test_final = pd.concat([X_test.drop(columns=obj_cols), encoded_df], axis=1)`

2. 여러 모델을 반복문으로 학습시켜 성능 비교 후 최적 모델 선택

```
for model in models:
    model.fit(X_train_final, y_train_encoded)
    y_pred_train = model.predict(X_train_final)
    model_accuracy = accuracy_score(y_train_encoded, y_pred_train)
    if model_accuracy > best_accuracy:
        best_accuracy = model_accuracy
        best_model = model
```

3. 예측 후 다시 원래 라벨로 역매핑하여 제출 형식 맞추기

```
reverse_mapping = {v: k for k, v in mapping.items()}
y_pred_test1['health_condition'] = y_pred_test1['health_condition'].map(reverse_mapping)
```

<새롭게 알게 된 내용>

train 시점에 fit 한 imputer, scaler, ohe 객체를 그대로 재사용해서 test/실제 제출 데이터에 transform 만 적용하는 패턴이 인상적이여서 새로웠음.

<배운 점>

OneHotEncoder(sparse_output=False) + .set_output(transform="pandas") 조합을 쓰면 인코딩 결과를 바로 df 으로 받을 수 있어서 pd.concat 이 훨씬 깔끔해짐. 예전처럼 .
수동 컬럼명 지정을 안 해도 됨

<어려운 점>

대회 평가지표가 Balanced Accuracy 인데 노트북 안에서는 일반 accuracy_score 만
사용했음. 클래스 불균형이 있다면 지표 불일치로 실제 리더보드 성능과 괴리가 생길 수
있음